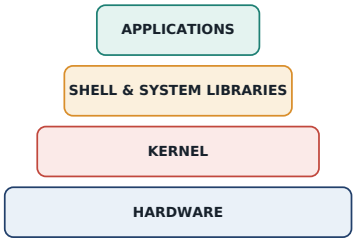
INTERVIEW PREP · HINGLISH EDITION

Operating Systems Interview Notes

Fresher se Advanced tak — har sawaal ka sabse aasaan jawab, samjhaya gaya Hinglish mein, diagrams ke saath.



Fresher Intermediate Advanced

54 Questions · 14 Diagrams · 3 Levels

CONTENTS

What's inside

How to use these notes

Har question ek chhota, self-contained card hai — question bold mein, uske neeche sabse simple Hinglish explanation, aur jaha zaroorat hai wahan ek diagram. Interviewer aksar ek topic (deadlock ya scheduling) pakad ke follow-up karte hain, isliye in explanations ko apne shabdon mein bolne ki practice zaroor karo — sirf padhna kaafi nahi hai.

FRESHER · 12 QUESTIONS

OS Basics & Process Fundamentals

The questions every fresher gets asked in round one — OS purpose, process vs. thread, kernels, system calls, and the process lifecycle.

F1 What is an OS? What are its main functions?

`OS basics`

OS ek system software hai jo hardware aur user applications ke beech **bridge** ka kaam karta hai. Ye 4 core kaam karta hai: **Process Management** (kaun kab CPU use karega), **Memory Management** (RAM allocate/deallocate), **File & Device Management** (files aur printer/disk jaise peripherals handle karna), aur **Security & Access Control** (resources ko unauthorized access se bachana). Simple line: bina OS ke, hardware sirf ek dabba hai jise koi bhi program directly control nahi kar sakta.

F2 Difference between a program, a process, and a thread

`process` `thread`

Program ek passive cheez hai — disk pe pada hua code, jo kuch nahi kar raha. **Process** uska running instance hai — execute hote hi OS use memory, PCB aur resources deta hai. **Thread** process ke andar ka lightweight execution unit hai — ek hi process ke multiple threads memory (code, data) share karte hain but apna khud ka stack aur register rakhte hain. Yaad rakho: Program = recipe, Process = us recipe se banaya khana (apni kitchen ke saath), Thread = ek hi kitchen me kaam karne wale multiple cooks.

F3 Multitasking vs multiprogramming vs multithreading

`scheduling basics`

Multiprogramming: multiple programs RAM me rakhe jaate hain taaki CPU kabhi idle na baithe — jab ek process I/O wait kare, CPU doosre pe switch ho jaye (sirf CPU utilization maximize karna hai). **Multitasking**: multiple tasks fast context-switching se “seemingly” ek saath chalte dikhte hain, user ko lagta hai sab parallel ho raha hai. **Multithreading**: ek hi process ke andar multiple threads simultaneously chalte hain (jaise ek app ke andar hi parallel kaam).

F4 Types of OS (batch, time-sharing, real-time, distributed, network)

`OS types`

Batch OS: jobs ko group me collect karke bina interaction ke ek-ek karke process karta hai. **Time-Sharing OS**: CPU time chhote time-slices me baant kar sabko turant response ka ehsaas deta hai. **Real-Time OS**: fixed deadline ke andar response guarantee karta hai — hard RTOS (deadline miss = failure, jaise missile control) vs soft RTOS (thoda delay chalta hai, jaise streaming). **Distributed OS**: multiple independent computers ek single system jaisa dikhte hain. **Network OS**: har machine apna OS rakhti hai but network ke through resources share karti hai.

F5 What is the kernel? Kernel vs shell

`kernel`

Kernel OS ka core/heart hai — hardware ke sabse close level pe memory, CPU scheduling aur device drivers jaise critical kaam handle karta hai. **Shell** ek interface hai (CLI/GUI) jo user ke commands leta hai aur unhe system calls ke through kernel tak pahunchata hai. Yaad rakho: Shell = waiter (order leta hai), Kernel = chef (asal kaam karta hai).

F6 Monolithic kernel vs microkernel

`kernel design`

Monolithic kernel me saari services (file system, drivers, memory management) ek hi bade kernel address space me chalti hain — fast hota hai but ek driver crash poora system crash kar sakta hai (Linux). **Microkernel** me sirf bare-minimum (IPC, basic scheduling) kernel space me hota hai, baaki services (drivers, file system) alag user-space processes ki tarah chalti hain — zyada stable/secure but IPC overhead ki wajah se thoda slow (Minix, QNX).

F7 What is a system call? Give examples

system calls · fork · exec · wait

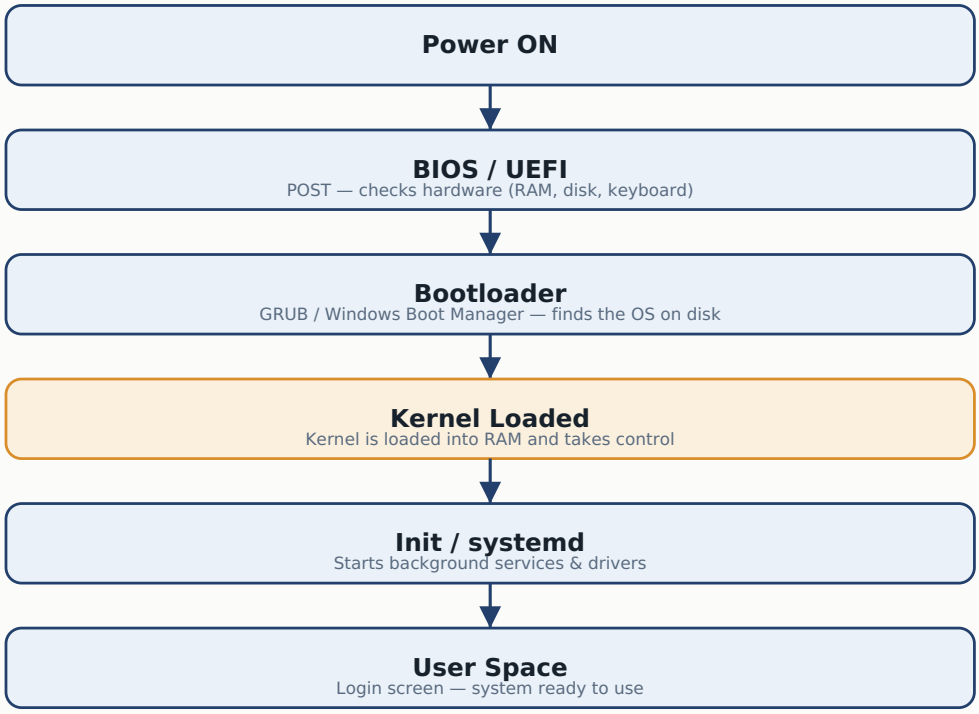
System call ek gateway hai jisse user program kernel se protected operation karwata hai — jaise file read/write, memory allocate, ya naya process banana. Common examples: **fork()** (naya child process banata hai), **exec()** (current process me naya program load karta hai), **wait()** (parent, child ke terminate hone ka wait karta hai), **exit()** (process terminate karta hai). Ye "user mode → kernel mode" switch trigger karta hai.

F8 What happens during the boot process?

boot · BIOS · bootloader

Power ON hote hi **BIOS/UEFI** POST (hardware check) karta hai, phir **bootloader** (GRUB / Windows Boot Manager) disk pe OS dhoondh kar **kernel** ko RAM me load karta hai. Kernel control lekar **init/systemd** process start karta hai jo background services chalata hai, aur aakhir me user ko login screen milti hai. Easy line: BIOS hardware check karta hai, bootloader OS dhoondh ke launch karta hai, kernel control le leta hai.

DIAGRAM

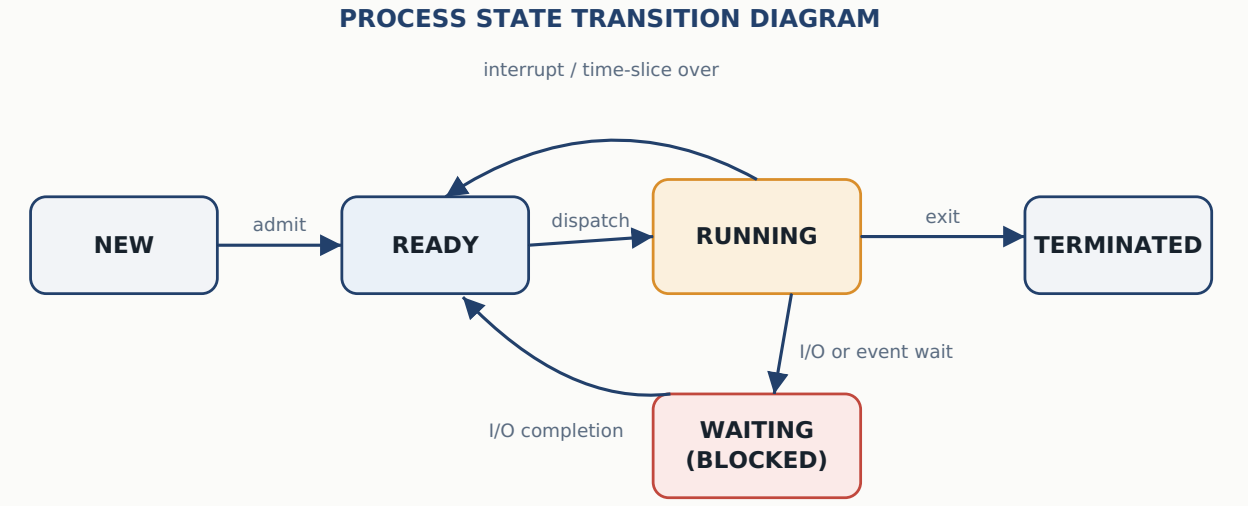


F9 Different process states (new, ready, running, waiting, terminated)

process states · PCB

5 main states hote hain: **New** (process create ho raha hai), **Ready** (CPU milne ka wait kar raha hai), **Running** (CPU pe actually execute ho raha hai), **Waiting/Blocked** (I/O ya kisi event ka wait kar raha hai), aur **Terminated** (execution complete, resources release ho rahe hain).

DIAGRAM



F10 What is a Process Control Block (PCB)? What does it store?

PCB

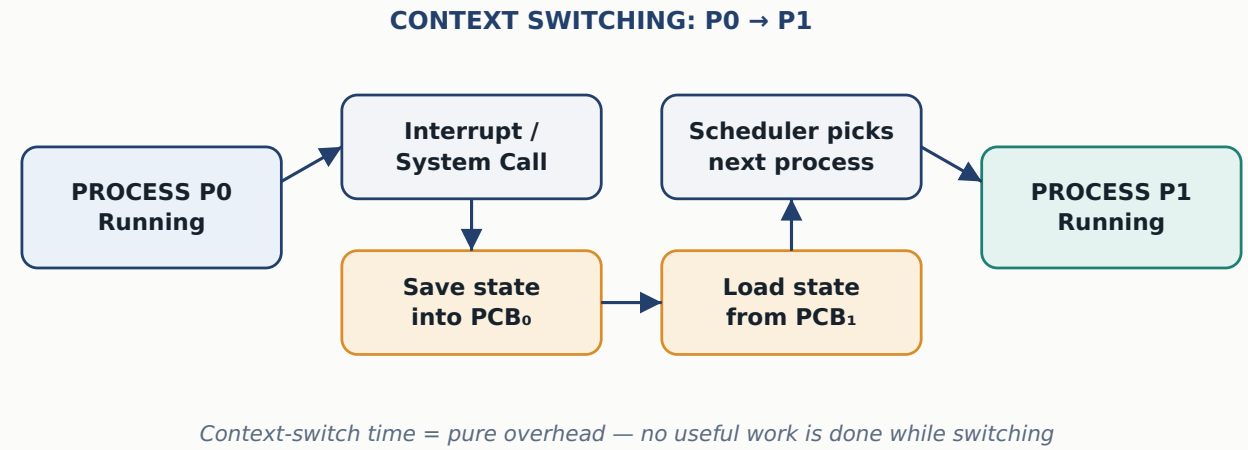
PCB OS ka ek data structure hai jo har process ki saari info store karta hai — process ID, state, program counter, CPU registers, memory limits, open files list, aur scheduling info. Jab context switch hota hai, OS current process ka PCB save karta hai aur next process ka PCB load karta hai — isi wajah se process apni pichli progress yaad rakh pata hai.

F11 What is context switching?

context switch · overhead

Context switching wo mechanism hai jisme CPU ek process se hatke doosre process pe switch karta hai — current process ka state PCB me save hota hai, phir next process ka saved state PCB se load hota hai. Ye **pure overhead** hai (isme koi useful kaam nahi hota, sirf switching cost hai), isliye OS ise minimize karne ki koshish karta hai.

DIAGRAM



Context-switch time = pure overhead — no useful work is done while switching

F12 User-level thread vs kernel-level thread

threads

User-level threads ek user-space library manage karti hai (kernel ko pata hi nahi) — fast create/switch hota hai but ek thread block hone par pura process block ho sakta hai. **Kernel-level threads** directly OS manage karta hai — thoda slow (kernel mode switch chahiye) but true parallelism milta hai multi-core pe, aur ek thread block hone par baaki thread chal sakte hain.

INTERMEDIATE · 21 QUESTIONS

Scheduling, Synchronization & Memory

Where most technical rounds live — CPU scheduling algorithms, race conditions, deadlocks, paging, virtual memory, and file systems.

I1 What is CPU scheduling? Why is it needed?

scheduling

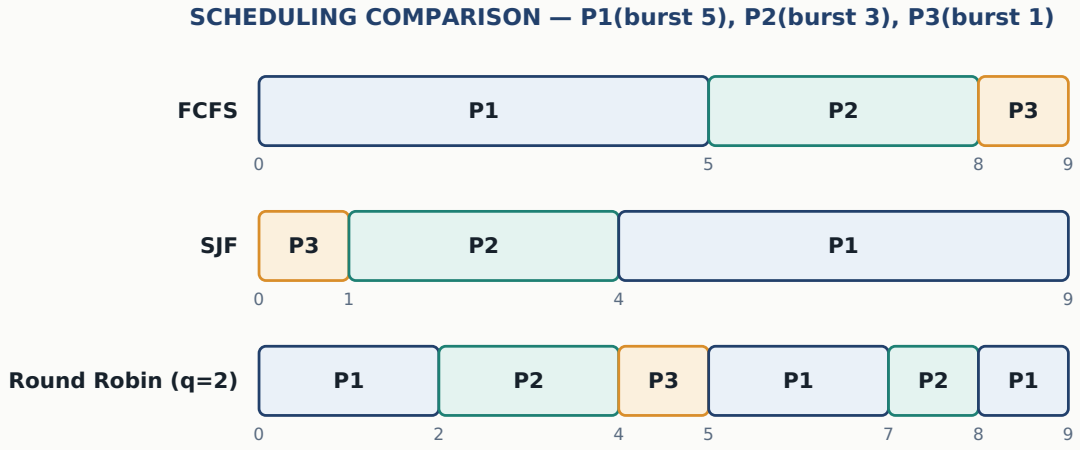
CPU scheduling wo decision-making process hai jisse OS decide karta hai ki Ready queue me se konsa process next CPU milega. Zaroorat isliye padti hai kyunki multiple processes hamesha CPU ke liye compete karte hain, aur goal hota hai **CPU utilization maximize** karna, **waiting time minimize** karna, aur fairness + responsiveness dena.

I2 Explain FCFS, SJF, Round Robin, Priority, and Multilevel Queue Scheduling

FCFS SJF Round Robin Priority MLQ

FCFS: jo pehle aaya wo pehle serve hoga — simple but convoy effect ka shikaar. **SJF**: sabse chhoti burst time wale process ko pehle chalao — average waiting time minimum but starvation ho sakti hai lambe process ke liye. **Round Robin**: har process ko ek fixed time quantum milta hai, phir queue ke end me — fair aur time-sharing ke liye best. **Priority Scheduling**: highest priority pehle chalta hai, low priority starve kar sakta hai (fix: aging). **Multilevel Queue**: processes ko category ke hisaab se alag queues me baanta jata hai, har queue ka apna algorithm ho sakta hai.

DIAGRAM



I3 Preemptive vs non-preemptive scheduling

preemptive

Preemptive: OS running process ko beech me rok ke doosre ko CPU de sakta hai (Round Robin, Priority with preemption) — responsive hota hai but context-switch overhead badhta hai. **Non-preemptive**: ek baar process CPU le le, khud terminate ya wait state me jaaye tabhi chhodta hai (FCFS, non-preemptive SJF) — simple but ek lamba process baaki sabko block kar sakta hai.

I4 Turnaround time, waiting time, and response time

scheduling metrics

Turnaround Time = Completion Time − Arrival Time (process ka poora safar). **Waiting Time** = Turnaround Time − Burst Time (Ready queue me kitni der baitha raha). **Response Time** = pehli baar CPU milne ka time − Arrival Time (interactive systems me sabse important — user ko jaldi “kuch ho raha hai” dikhna chahiye).

I5 What is convoy effect (in FCFS)?

FCFS convoy effect

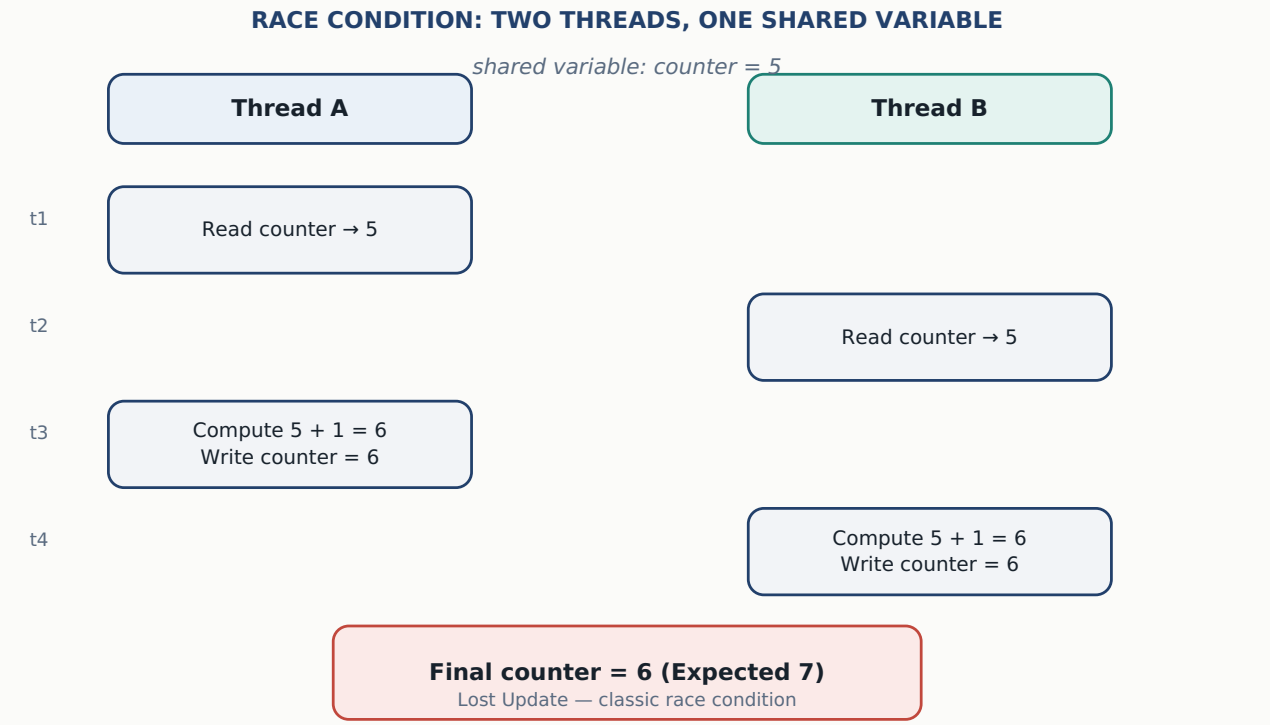
Convoy effect FCFS me hota hai — agar ek lamba (CPU-heavy) process pehle queue me aa gaya, toh uske peeche saare chhote processes CPU ke liye fasi rehte hain, jaise ek slow truck ke peeche traffic jam. Average waiting time badh jata hai. Solution: SJF ya Round Robin use karo.

I6 What is a race condition?

race condition synchronization

Race condition tab hoti hai jab do ya zyada processes/threads ek shared resource ko simultaneously access/modify karte hain, aur final result is baat pe depend karta hai ki kaun pehle execute hua — outcome unpredictable ho jata hai. Classic example: do threads ek hi counter pe `counter++` kar rahe hain bina synchronization ke; final value expected se kam aa sakti hai.

DIAGRAM



I7 What is the critical section problem? Conditions for a valid solution

critical section

Critical section wo code ka part hai jaha shared resource access hota hai. Valid solution ke liye 3 conditions chahiye: **Mutual Exclusion** (ek time pe sirf ek process andar ho sakta hai), **Progress** (koi andar nahi hai toh entry chahne wale ko bina wajah rokna nahi), aur **Bounded Waiting** (ek process ko infinite wait nahi karna — fair chance milna chahiye).

I8 Mutex vs semaphore

mutex semaphore

Mutex sirf lock/unlock karta hai — ek time pe sirf ek thread lock hold karta hai, aur jisne liya wahi unlock kar sakta hai (ownership hoti hai). **Semaphore** ek integer counter hai jo `wait()` / `signal()` se control hota hai — multiple resources manage kar sakta hai, ownership zaroori nahi. Simple line: Mutex = ek hi chaabi wala tala, Semaphore = N-slot wala parking lot.

I9 Binary semaphore vs counting semaphore

semaphore

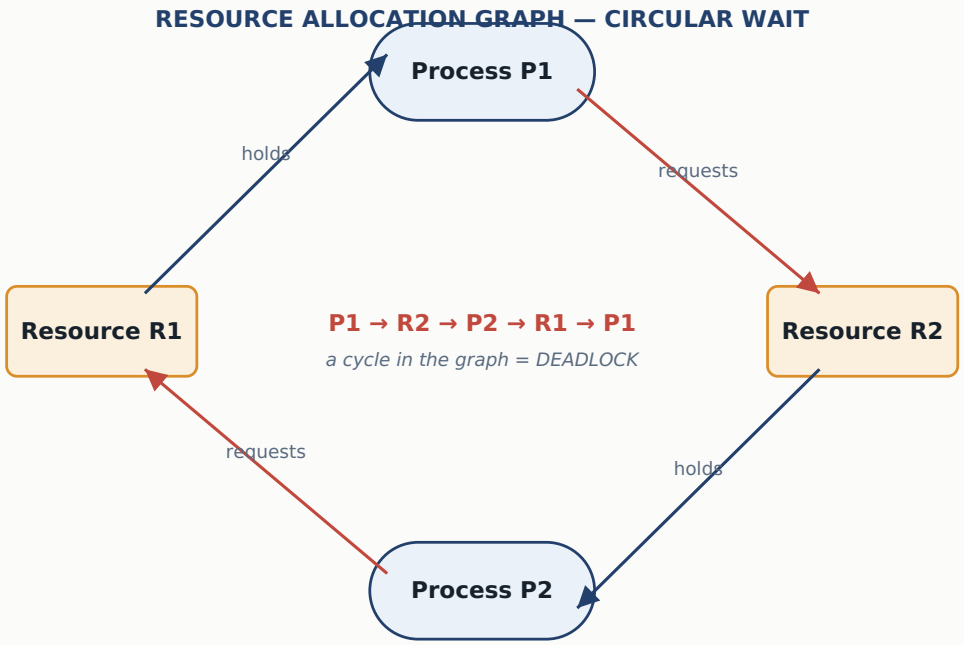
Binary semaphore sirf 0 ya 1 leta hai — mutex jaisa behave karta hai, ek resource ke liye. **Counting semaphore** koi bhi non-negative integer le sakta hai — multiple instances wale resource manage karta hai (jaise 3 printers wale pool ka semaphore 3 se start hoga).

I10 What is deadlock? Four necessary conditions

deadlock · Coffman conditions

Deadlock ek situation hai jaha do ya zyada processes ek doosre ke resource release hone ka wait kar rahe hain, aur koi aage nahi badh pata. 4 necessary conditions (Coffman): **Mutual Exclusion** (resource sirf ek process use kar sakta hai), **Hold and Wait** (resource pakde hue nayi resource ka wait), **No Preemption** (resource forcefully chheena nahi ja sakta), **Circular Wait** (processes circular chain me ek doosre ka wait kar rahe hain). Deadlock tabhi hota hai jab chaaron ek saath ho.

DIAGRAM



I11 Deadlock prevention vs avoidance vs detection vs recovery

deadlock handling

Prevention: 4 conditions me se ek ko structurally todna (jaise resources ek fixed order me maango — circular wait khatam). **Avoidance**: dynamically check karo ki koi allocation unsafe state me toh nahi le ja raha (Banker's Algorithm). **Detection**: deadlock hone do, phir periodically resource graph check karke detect karo. **Recovery**: detect hone ke baad process kill karo ya resource forcefully preempt karo taaki system aage badh sake.

I12 What is the Banker's Algorithm?

Banker's algorithm · deadlock avoidance

Ek deadlock-avoidance technique jo har naye resource request ko grant karne se pehle check karta hai ki kya system is allocation ke baad bhi **safe state** me rahega (ek aisi sequence exist karti hai jisme sab processes complete ho sakein). Agar request unsafe state me le jaye, use taal diya jata hai. Naam “Banker's” isliye kyunki bank ki tarah loan (resource) tabhi deta hai jab pakka ho sab depositors ko wapas mil jayega.

I13 Contiguous vs non-contiguous memory allocation

memory allocation

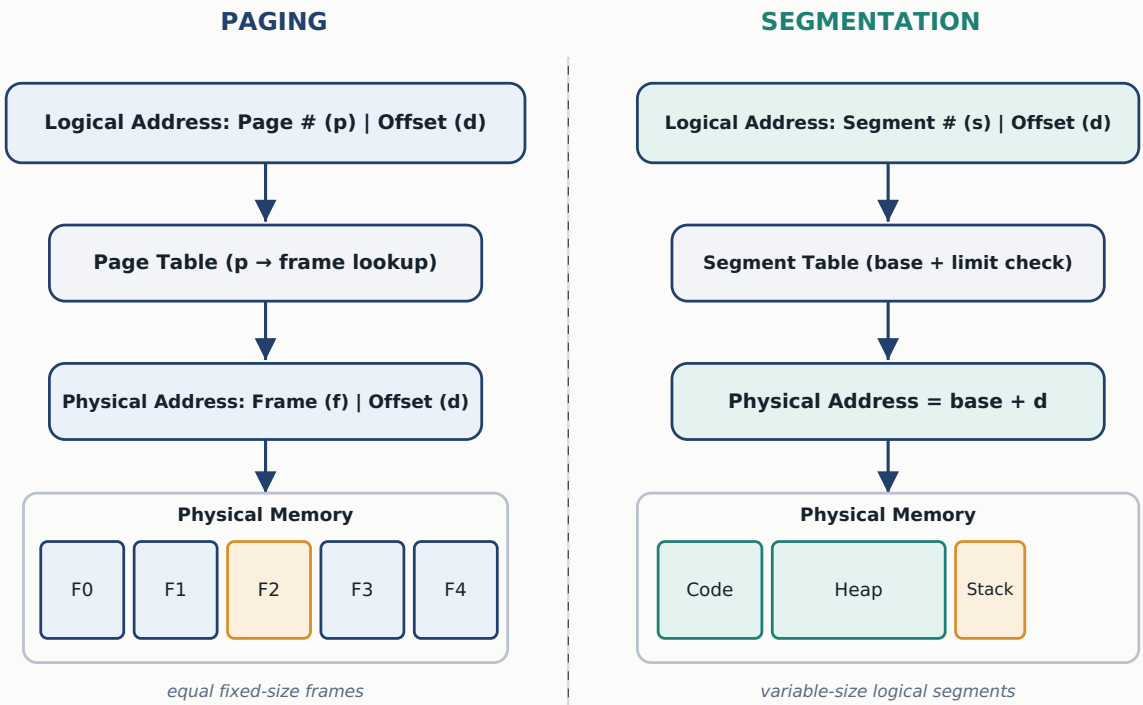
Contiguous: process ko memory ka ek single continuous block milta hai — access fast but external fragmentation ka problem (chhote-chhote free blocks bikhre reh jaate hain). **Non-contiguous** (paging/segmentation): process ke parts alag-alag jagah fit ho sakte hain — fragmentation kam but address translation ke liye extra data structure (page table) chahiye.

I14 What is paging? What is segmentation? Paging vs segmentation

paging segmentation

Paging memory ko fixed-size pages (logical) aur frames (physical) me divide karta hai — koi bhi page kisi bhi free frame me fit ho sakta hai, external fragmentation khatam ho jata hai. **Segmentation** memory ko variable-size logical units (segments) me divide karta hai jo program structure (code, stack, heap) reflect karte hain — zyada natural but external fragmentation wapas aa sakta hai kyunki segments variable size ke hote hain.

DIAGRAM



I15 Internal fragmentation vs external fragmentation

fragmentation

Internal fragmentation tab hoti hai jab allocated block process ki need se bada ho — bacha space waste ho jata hai kyunki wo kisi aur ko allot nahi ho sakta (fixed-size/paging me common). **External fragmentation** tab hoti hai jab total free memory enough hai but scattered chhote pieces me hai, ek bade continuous request ke liye use nahi hoti (contiguous allocation me common).

I16 What is virtual memory and why do we need it?

virtual memory

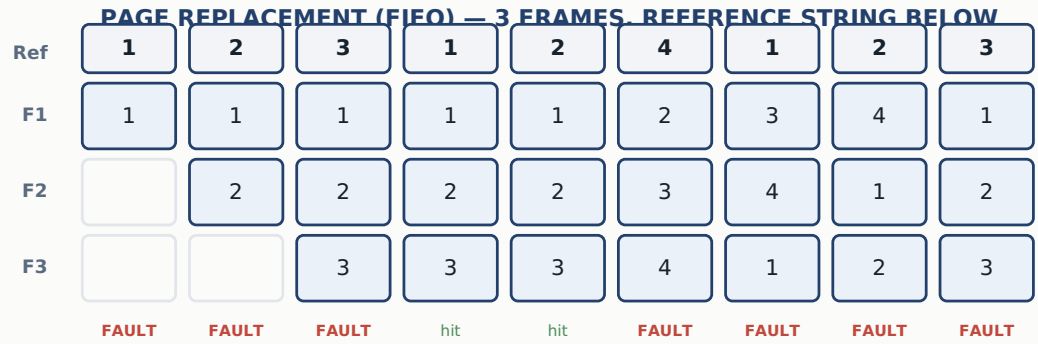
Virtual memory ek technique hai jisse process ko lagta hai uske paas apni khud ki bahut badi, continuous memory hai — chahe actual RAM kitni bhi choti ho. Demand paging use karke sirf zaroori pages RAM me rakhe jaate hain, baaki disk (swap space) pe rehte hain. Fayda: bade programs kam RAM me bhi chal sakte hain, aur processes ek doosre ki memory se isolated rehte hain (protection).

I17 Page replacement algorithms: FIFO, LRU, Optimal

`page replacement`

Jab RAM full ho aur naya page load karna ho, OS ko decide karna padta hai kaunsa purana page hatana hai. **FIFO**: sabse purana loaded page hatao (simple but Belady's anomaly ho sakti hai). **LRU**: jo page sabse zyada der se use nahi hua use hatao (real usage pattern ke close, practically bahut use hota hai). **Optimal**: future me sabse door use hone wala page hatao (theoretically best but future predict nahi kar sakte, sirf benchmark ke liye).

DIAGRAM



Total page faults = 7 out of 9 references

I18 What is thrashing? How is it avoided?

`thrashing`

Thrashing tab hoti hai jab CPU apna zyadatar time actual kaam ke bajaye pages ko RAM aur disk ke beech swap karne me spend karta hai — CPU utilization drastically gir jata hai jabki system “busy” dikhta hai. Ye over-multiprogramming se hota hai. Solution: degree of multiprogramming kam karo, ya working-set model / page-fault-frequency monitoring use karo.

I19 What is a file system? Basic file operations

`file system`

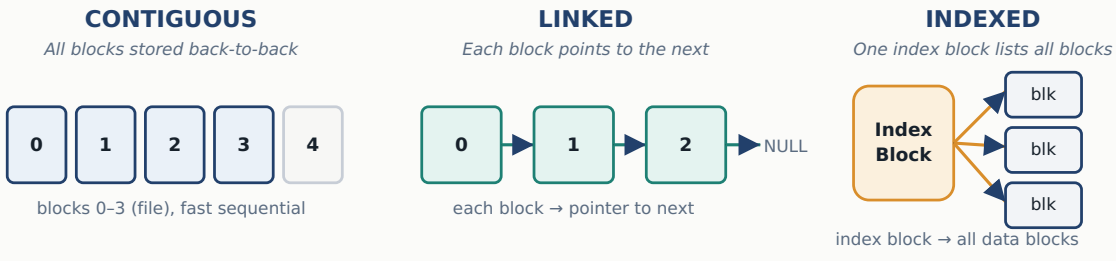
File system OS ka wo part hai jo decide karta hai data disk pe kaise store, organize, aur retrieve hoga — files ko directories me structure karta hai. Basic operations: create, open, read, write, close, delete, rename. Metadata bhi maintain karta hai (size, permissions, timestamps, location on disk).

I20 File allocation methods: contiguous, linked, indexed

`file allocation`

Contiguous: file ke saare blocks disk pe consecutively store hote hain — fast sequential access but fragmentation aur growth ka problem. **Linked**: har block agle block ka pointer store karta hai — fragmentation nahi hoti but random access slow hai. **Indexed**: ek alag index block saare data blocks ke addresses store karta hai — random access fast, modern inode-based file systems isi ka use karte hain.

DIAGRAM



I21 What is a file descriptor?

`file descriptor`

File descriptor ek chhota non-negative integer hai jo OS kisi open file ko represent karne ke liye process ko deta hai — is number se read/write/close operations hoti hain bina poora path baar-baar bataye. Har process ke paas by default 3 descriptors hote hain: 0 (stdin), 1 (stdout), 2 (stderr).

ADVANCED · 21 QUESTIONS

Deep Internals & Systems Design

For senior / SDE-2+ rounds — classic synchronization problems, TLBs, real-time scheduling, disk scheduling, RAID, and whiteboard walkthroughs.

A1 Inter-process communication methods: pipes, message queues, shared memory, sockets

IPC

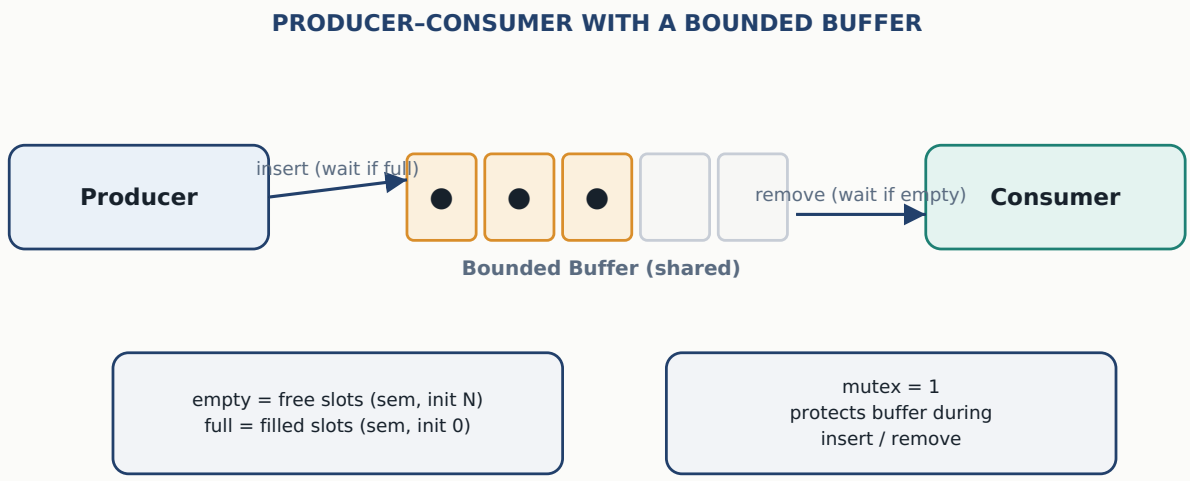
Pipes: ek-directional data stream, related processes (parent-child) ke beech. **Message Queues**: structured messages queue me daalo, koi bhi process padh sakta hai, order maintain hota hai. **Shared Memory**: sabse fast — common memory segment jo multiple processes access karte hain (synchronization khud handle karni padti hai). **Sockets**: network ke through communication, same machine ya distributed systems dono me.

A2 Producer-consumer, readers-writers, and dining philosophers problems

classic sync problems

Producer-Consumer: producer shared bounded buffer me data daalta hai, consumer nikalta hai — buffer full ho toh producer wait, empty ho toh consumer wait; do counting semaphores (empty, full) + ek mutex se solve hota hai. **Readers-Writers**: multiple readers ek saath padh sakte hain but writer ko exclusive access chahiye — reader-count track hota hai. **Dining Philosophers**: 5 philosophers, 5 forks — deadlock avoid karne ke liye classic solution: ek philosopher ulta order me fork uthaye (asymmetric solution), taaki circular wait na bane.

DIAGRAM



A3 Monitors vs semaphores

monitors

Monitor ek high-level construct hai jisme shared data + us pe operate karne wale procedures ek hi structure me bundled hote hain, aur monitor khud mutual exclusion ensure karta hai — condition variables (wait/signal) waiting ke liye use hoti hain. **Semaphore** lower-level tool hai jaha programmer ko khud wait()/signal() sahi jagah lagana padta hai — galti hone ke chances zyada, monitor zyada safe/structured hota hai kyunki runtime enforce karta hai.

A4 Spinlocks vs semaphores — when would you use each?

spinlock

Spinlock me waiting thread busy-wait karta hai (continuously check karte rehta hai lock free hua ya nahi) — CPU waste hota hai but agar lock thodi der ke liye hold hoga (multiprocessor kernel code), context-switch ki cost bachti hai. **Semaphore** me waiting thread ko block/sleep kar diya jata hai, CPU doosre process ko mil jata hai — single-processor ya lambi wait ke liye better.

A5 What is priority inversion? What is priority inheritance?

`priority inversion`

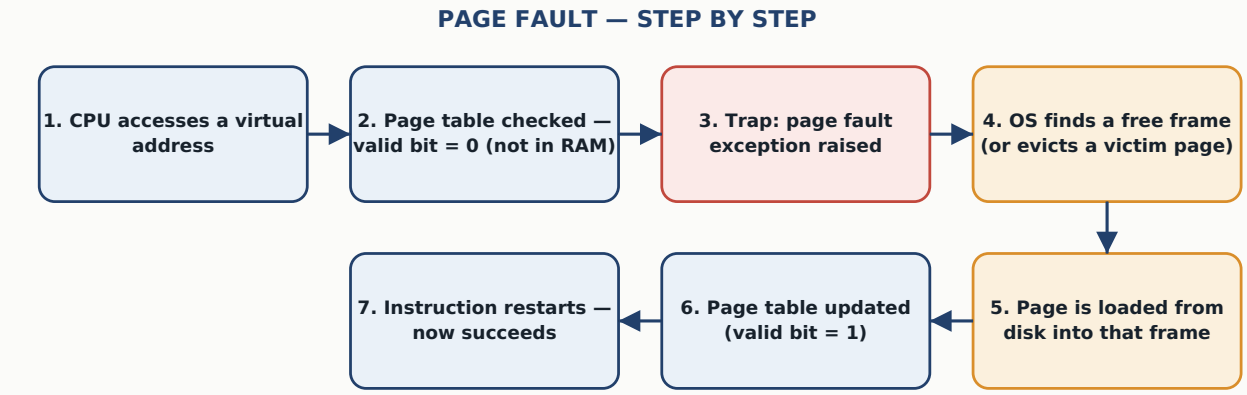
Priority inversion tab hota hai jab ek high-priority process ek low-priority process dwara hold kiye resource ka wait kar raha ho, aur beech me medium-priority process aake CPU le le — effectively high priority, medium se bhi peeche chala jata hai. **Priority Inheritance** isse solve karta hai: jab high-priority process resource ka wait kare, resource hold karne wale low-priority process ki priority temporarily badha di jaati hai taaki wo jaldi resource release kare.

A6 Demand paging and the page fault lifecycle

`demand paging` `page fault`

Demand paging me koi bhi page tabhi RAM me load hota hai jab uski actually zaroorat pade, poora process ek saath load nahi hota — memory efficient hota hai. Page fault lifecycle: CPU page access karta hai → page table check hota hai (valid bit) → page RAM me na ho toh trap generate hota hai → OS disk se page ko free frame me laata hai (zaroorat pade toh kisi ko replace karta hai) → page table update hota hai → instruction restart hoti hai.

DIAGRAM



A7 What is Belady's anomaly?

`Belady's anomaly` `FIFO`

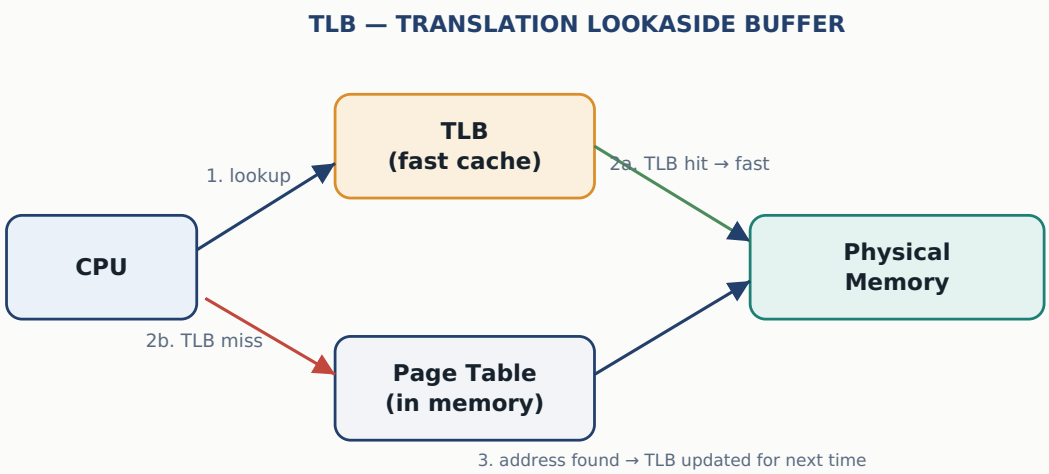
Ek counter-intuitive situation jisme page frames badhane par (zyada RAM dene par) bhi page faults kam hone ke bajaye badh jaate hain — ye FIFO page replacement me observe hota hai. Isi wajah se FIFO ko “stack algorithm” nahi mana jata, jabki LRU aur Optimal is anomaly se safe hote hain.

A8 Translation Lookaside Buffer (TLB)

TLB

TLB ek chhota, super-fast cache hai jo recently-used page table entries (virtual → physical address mapping) store karta hai, taaki har memory access pe poori page table walk na karni pade. TLB hit hone par translation almost instant hoti hai; TLB miss hone par CPU ko actual page table me jaake dekhna padta hai (slow), phir wo entry future ke liye TLB me daal deta hai.

DIAGRAM



A9 Inverted page tables

inverted page table

Normal page table har process ke liye alag table rakhta hai (jitne pages utni entries) jo bade address space me bahut memory le sakta hai. Inverted page table iska ulta karta hai — ek single system-wide table rakhta hai jisme har physical frame ke liye ek entry hoti hai, aur batata hai "ye frame kis process ke kis page ko hold kar raha hai" — memory bachti hai but search thoda slow hota hai (hashing se fast banaya jata hai).

A10 Copy-on-write — where is it used?

copy-on-write fork

Copy-on-Write (CoW) ek optimization hai jisme `fork()` se child process banne par turant parent ki memory copy nahi hoti — dono same physical pages share karte hain (read-only mark karke). Jaise hi koi ek process us page me kuch likhne ki koshish kare, tabhi actual copy banti hai. Isse fork() bahut fast ho jata hai, especially jab child turant exec() call karne wala ho.

A11 NUMA architecture and its impact on performance

NUMA

NUMA (Non-Uniform Memory Access) me har CPU/core ka apna "local" memory hota hai jise wo fast access kar sakta hai, jabki doosre CPU ki "remote" memory access karne me zyada time lagta hai — UMA se alag jaha sabhi CPUs ek shared memory equal speed se access karte hain. NUMA-aware scheduling important hai — process ko us CPU pe rakho jiske paas uski memory locally available hai.

A12 Multithreading models: many-to-one, one-to-one, many-to-many

multithreading models

Many-to-One: multiple user threads ek hi kernel thread pe map — fast (user-space management) but true parallelism nahi (ek blocking call sab rok deta hai). **One-to-One**: har user thread ka apna kernel thread — true parallelism multi-core pe, but har thread create karna kernel overhead leke aata hai. **Many-to-Many**: multiple user threads, multiple kernel threads pe map (flexible ratio) — best of both but implementation complex.

A13 Real-time scheduling: Rate Monotonic Scheduling, Earliest Deadline First

RMS EDF

Rate Monotonic Scheduling: fixed-priority algorithm — jitni chhoti period (jitni jaldi-jaldi task repeat hota hai), utni high priority — static priorities, periodic tasks ke liye. **Earliest Deadline First**: dynamic-priority — jis task ki deadline sabse jaldi aa rahi hai use highest priority (priority runtime me change hoti hai) — EDF theoretically zyada CPU utilization achieve kar sakta hai RMS se (up to 100%).

A14 Soft real-time vs hard real-time systems

real-time systems

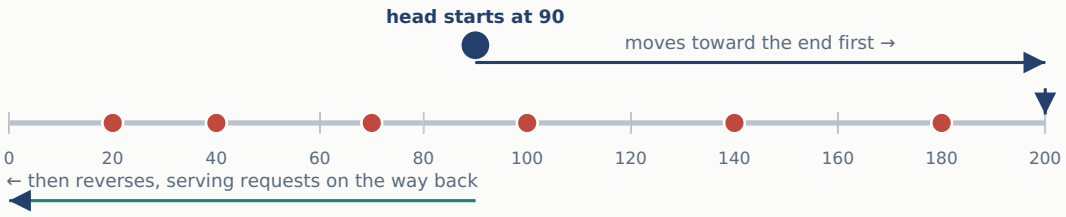
Hard real-time me deadline miss hona critical failure hai — jaise missile guidance ya pacemaker, ek bhi miss disaster ban sakta hai. **Soft real-time** me deadline miss acceptable hai (though undesirable) — quality thodi degrade hoti hai but system crash nahi hota, jaise video streaming ka ek frame late aana.

A15 Disk scheduling: FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK

disk scheduling

FCFS: jis order me requests aayi usi order me serve — seek time zyada ho sakta hai. **SSTF**: head ke sabse paas wali request pehle — efficient but door wali requests starve ho sakti hain. **SCAN** ("elevator algorithm"): head ek direction me end tak jaata hai sab serve karte hue, phir wapas mudta hai. **C-SCAN**: end tak jaake wapas seedha shuru me jump — wait time zyada uniform. **LOOK/C-LOOK**: SCAN/C-SCAN jaisa but end tak jaane ke bajaye sirf last request tak jaake mudte hain — unnecessary travel bachta hai.

DIAGRAM



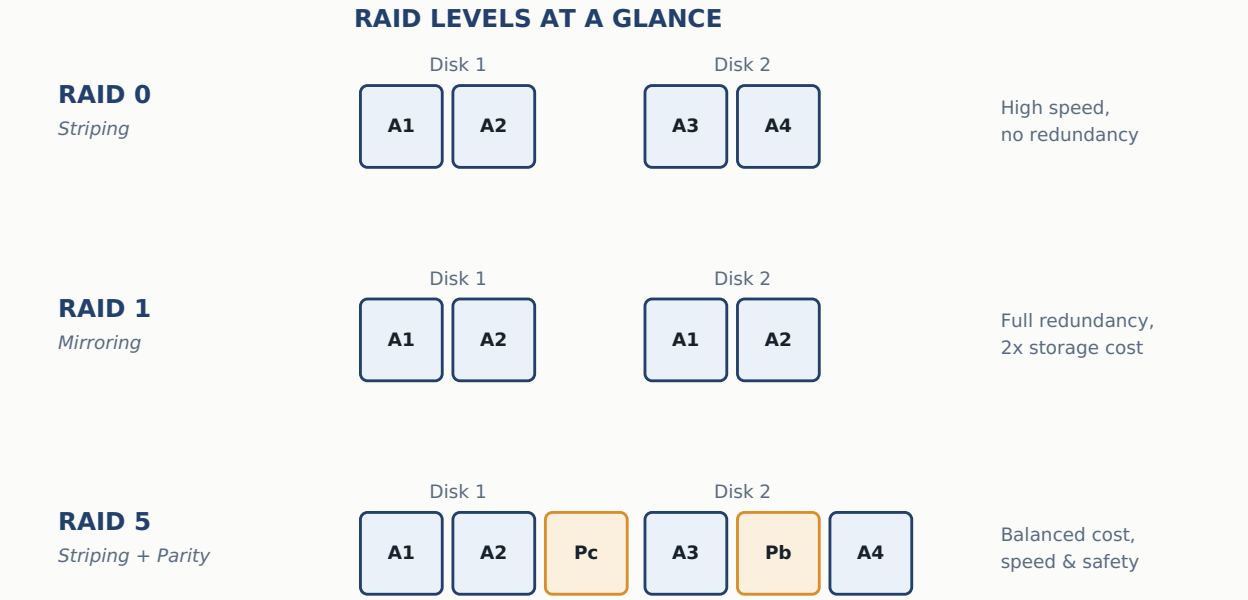
Requests served in order: 100, 140, 180, (end), 70, 40, 20 — like a lift serving floors in one direction at a time

A16 RAID levels (0, 1, 5, 10) — trade-offs

RAID

RAID 0 (Striping): data multiple disks me split parallel likha/padha jata hai — speed bahut zyada but redundancy zero. **RAID 1** (Mirroring): data ka exact copy do disks pe — ek fail ho toh doosre se recover, but storage cost double. **RAID 5** (Striping + Parity): data aur parity multiple disks me distribute — ek disk fail ho toh parity se reconstruct, achha balance. **RAID 10**: RAID 1+0 — mirror pehle phir stripe — speed aur redundancy dono, cost sabse zyada.

DIAGRAM



A17 Journaling file systems — why are they crash-safe?

journaling ext4 NTFS

Journaling file system disk pe changes karne se pehle unhe ek “journal” (log) me likh leta hai — pehle intent likha jata hai, phir actual change hoti hai. Beech me crash ho jaye (power failure), reboot ke baad OS journal check karke incomplete operations ko safely complete ya rollback kar deta hai — isse corruption se bachta hai aur lambi manual fsck check ki zaroorat nahi padti (ext4, NTFS isi ka use karte hain).

A18 How does an interrupt get handled by the OS?

interrupts ISR

Jab koi hardware device (keyboard, disk, timer) ko CPU ka attention chahiye, wo ek interrupt signal bhejta hai. CPU current instruction complete karke apna state save karta hai, phir **Interrupt Service Routine (ISR)** — ek chhota special function — execute karta hai, aur baad me apna pehla kaam (saved state se) resume kar deta hai. Interrupts hi OS ko polling (baar-baar check karte rehna) se bachate hain.

A19 Cache coherence problem in multiprocessor systems

cache coherence MESI

Multi-core systems me har core ka apna local cache hota hai, aur agar ek core kisi shared variable ki value apne cache me update kare, doosre cores ke paas usi variable ki purani (stale) value reh sakti hai — ye cache coherence problem hai. Isse solve karne ke liye protocols use hote hain jaise **MESI** (Modified, Exclusive, Shared, Invalid) jo track karte hain data ka konsa version kaha valid hai.

A20 Hypervisors: Type 1 vs Type 2 virtualization

`hypervisor`

Type 1 (“bare-metal”) directly hardware pe install hota hai, bina host OS ke — fast aur efficient, data centers me use hota hai (VMware ESXi, Xen). **Type 2** (“hosted”) ek normal OS ke upar application ki tarah install hota hai (VirtualBox, VMware Workstation) — use karna aasan but extra layer ki wajah se thoda slower.

A21 How does the OS handle a page fault, step by step? (common whiteboard question)

`page fault` `whiteboard`

1. CPU ek virtual address access karta hai. **2.** Page table check hota hai — valid bit 0 (page RAM me nahi). **3.** Trap (page fault exception) generate hota hai. **4.** OS address valid hai ya illegal check karta hai. **5.** Valid ho toh free frame dhoonda jata hai (zaroorat pade toh page replacement se victim evict, dirty ho toh pehle disk pe likha jata hai). **6.** Zaroori page disk se us frame me load hota hai. **7.** Page table update (valid bit=1). **8.** Process “ready” me wapas aata hai aur wahi instruction dobara try hoti hai — is baar successfully. (See the diagram under Q A6 for the same flow visually.)

FINAL REVISION

Quick Revision Cheat Sheet

Skim this page 10 minutes before you walk into the interview.

Turnaround Time	Completion Time − Arrival Time
Waiting Time	Turnaround Time − Burst Time
Response Time	Time of first CPU burst − Arrival Time
4 Deadlock Conditions	Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait
Deadlock handling	Prevention → Avoidance → Detection → Recovery
Critical Section rules	Mutual Exclusion, Progress, Bounded Waiting
Mutex vs Semaphore	Mutex = lock with ownership | Semaphore = counter, no ownership
Page replacement	FIFO (simple) < LRU (practical) < Optimal (theoretical benchmark)
Paging vs Segmentation	Paging = fixed-size frames | Segmentation = variable-size, logical
RAID 0 / 1 / 5 / 10	Striping / Mirroring / Striping+Parity / Mirror+Stripe
Disk scheduling	FCFS → SSTF → SCAN → C-SCAN → LOOK → C-LOOK
Monitor vs Semaphore	Monitor = built-in mutual exclusion | Semaphore = manual wait/signal

All the best for your interview — jaake tod do! ★